



Architecting Trustworthy AI for Systems Engineering Cost Estimation

Ray Madachy
Naval Postgraduate School

INCOSE International Symposium 2026
Yokohama, Japan | June 13–18, 2026



Introduction and Background

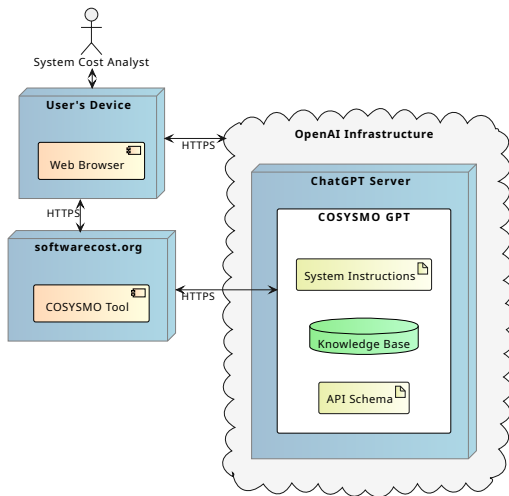
- Large language models (LLMs) are powerful for natural language understanding, but unreliable for exact calculations or domain-specific quantitative models.
- Cost estimation in systems engineering requires deterministic, transparent results.
- Hallucinations undermine trust in AI-driven workflows.
- Goal is to use Generative AI to help compose model inputs and bind LLMs to trusted external tools for guaranteed correctness [1].
- Implemented for the Constructive Systems Engineering Cost Model (COSYSMO) [2] with the COSYSMO GPT [3]

Approach

- Restrict LLM from internal mathematical computation.
- Delegate all quantitative model calculations to trusted COSYSMO Tool [4] via API.
- Enforce deterministic invocation through structured endpoints.
- Implement multi-modal Generative AI for smart input inference with natural language and images for cost parameter extraction.
- Ensure transparency and traceability of outputs.

Deployment Architecture

- Architecture includes user interaction, AI mediation, and deterministic computational services.
- LLM mediation governed by system instructions, domain knowledge base, and formally defined API schema.
- All authoritative computation delegated to independently validated external tool.
- Pattern generalizes to other quantitative SE domains [1].



COSYSMO Effort Equation

$$Effort = A \cdot \left(\sum_k (w_e \Phi_{e,k} + w_n \Phi_{n,k} + w_d \Phi_{d,k}) \right)^B \cdot \prod_{j=1}^{14} EM_j$$

Where

- Effort is in Person-Months (PM) for a nominal schedule
- A is a constant derived from historical project data
- k = Numbers of Requirements, Interfaces, Algorithms and Scenarios
- w_x = weight for easy (e), nominal (n), or difficult (d) size driver
- Φ is the quantity of “k” size driver
- B is an exponent for the diseconomy of scale
- EM_i is an effort multiplier for the i th cost driver. The geometric product is an overall Effort Adjustment Factor (EAF) to the nominal effort.

Requirements

- GPT helped develop its own requirements iteratively
- Fed back into its instructions in feedback loop

COSYSMO GPT v 1.0

1. User Interaction & Elicitation
 - 1.1 Elicit system size drivers (Easy, Nominal, Difficult)
 - 1.2 Elicit cost factor ratings for all COSYSMO drivers
 - 1.3 Accept partial input with defaults
 - 1.4 Solicit/accept labor rate (default \$20,000)
 - 1.5 Prompt clarifications for semantic correctness
2. Defaults, Validation & Normalization
 - 2.1 Default missing size fields to 0
 - 2.2 Default unspecified cost factors to Nominal
 - 2.3 Validate categorical entries against allowed values
 - 2.4 Normalize synonymous user inputs
 - 2.5 Reject/re-prompt nonsensical values
 - 2.6 Ensure URL-safe encoding
3. Input Rationale Table (Pre-API)
 - 3.1 Produce Markdown input table
 - 3.2 Populate rationale/assumption entries
 - 3.3 Ensure table is complete, consistent, ready for review
4. API Request Construction
 - 4.1 Construct GET URL with base endpoint
 - 4.2 Append all input fields as URL parameters
 - 4.3 Verify deterministic parameter order
 - 4.4 Log final resolved URL in transcript

Requirements (Cont.)

- 5. Retrieval & Parsing (Web Tool)
 - 5.1 Use web tool to dispatch GET request
 - 5.2 Parse and extract required fields
 - 5.2.1 Effort (Person-Months)
 - 5.2.2 Schedule (Months)
 - 5.2.3 Cost (USD)
 - 5.2.4 Total Size (Equivalent Nominal Requirements)
 - 5.2.5 Effort Distribution table
 - 5.2.6 Pie chart image in SVG format
 - 5.3 Attempt robust parsing with fallback selectors
 - 5.4 Detect HTTP errors and route to error handling
- 6. Output Composition & Presentation
 - 6.1 Display plain-text summary format
 - 6.2 Present Effort Distribution as clean table
 - 6.3 Present effort activity pie chart after table
 - 6.4 Preserve SVG content verbatim from source
 - 6.5 Ensure numeric outputs are properly rounded
- 7. Provenance, Transparency & Source Linking
 - 7.1 Append clickable hyperlink to exact URL used
 - 7.2 Confirm displayed results match linked page content
- 8. Error Handling & Resilience
 - 8.1 Handle unreachable tool with standard message
 - 8.2 Handle partial extraction failures gracefully
 - 8.3 Never fabricate results in case of errors
 - 8.4 Avoid repeated rapid retries (max one)

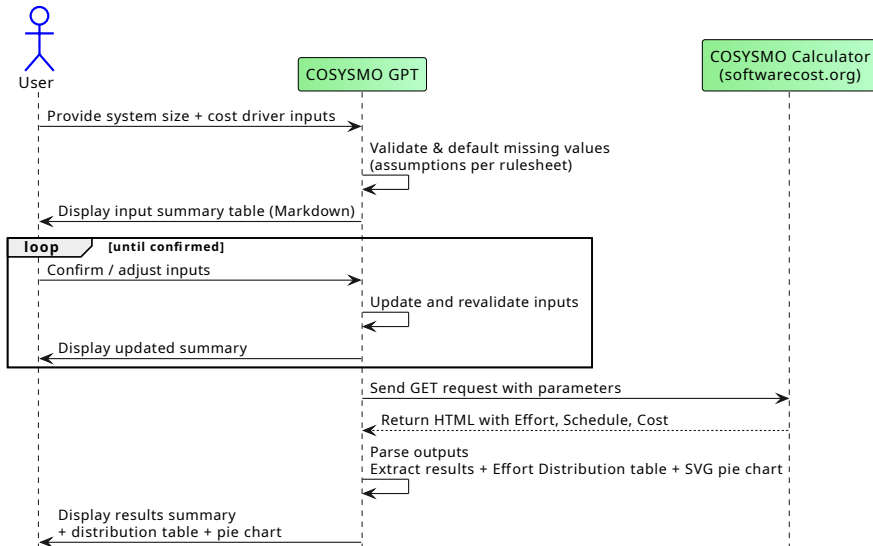
Requirements (Cont.)

- 9. GenAI/LLM Reasoning & Knowledge Use
 - 9.1 Conduct reasoning using trillion-token corpus
 - 9.2 Interpret ambiguous inputs using statistical regularities
 - 9.3 Propose default cost driver values from corpus patterns
 - 9.4 Draft rationales reflecting aggregated knowledge
 - 9.5 Maintain neutral academic tone with labeled assumptions
 - 9.6 Not infer proprietary data; use public corpus only
 - 9.7 Reflect COSYSMO semantics with fidelity
 - 9.8 Operate at scale infeasible for human experts
- 10. Security, Privacy & Compliance
 - 10.1 Avoid collecting sensitive personal data
 - 10.2 Not embed user-identifying tokens in URL
 - 10.3 Redact sensitive content from rationale
- 11. Tooling & Configuration Constraints
 - 11.1 Use only web tool for HTTP retrieval
 - 11.2 Operate at temperature = 0.3 (unless overridden)
 - 11.3 Refrain from non-approved tools for scraping
- 12. Non-Functional Requirements (Quality Attributes)
 - 12.1 Performance: Complete within acceptable window
 - 12.2 Reliability: Produce deterministic URLs
 - 12.3 Usability: Readable, well-aligned input table
 - 12.4 Maintainability: Single canonical parameter list
 - 12.5 Traceability: Evident input→URL→output flow
- 13. Verification & Validation
 - 13.1 Unit Verification: Confirm defaults and normalization
 - 13.2 Integration Verification: Validate URL construction and HTML parsing
 - 13.3 System Validation: End-to-end test cases
 - 13.4 Negative Tests: Error scenarios

Custom GPT Design

- Built using OpenAI Custom GPT framework with instructions and knowledge base.
- Configured with deterministic delegation rules.
- Inputs gathered from users in natural language, uploaded documents or pasted visual media.
- Missing values defaulted using structured assumptions.
- Outputs formatted with summary tables, input rationale, effort breakdowns, and diagrams.

Sequence Model Diagram



GPT Instructions

- Instructions constrain assistant behavior: elicitation rules, permissible assumptions, required tool usage, output formatting.
- Written in Markdown to support hierarchical organization, readability by both humans and machines, and structured operational logic.
- See example excerpt.

Core Rule

Never compute COSYSMO results internally. All quantitative model calculations **MUST** be delegated to the external COSYSMO tool via the estimateCosysmo operation. Do not approximate, round, or substitute internal arithmetic for tool output.

Input Collection

Elicit system size drivers (Requirements, Interfaces, Algorithms, Scenarios) at Easy / Nominal / Difficult levels. Collect all 14 cost driver ratings. Apply defaults: missing size fields = 0; unspecified cost factors = Nominal. Normalize synonymous inputs (e.g., "low risk" → Low for Technology Risk). Re-prompt for nonsensical values.

Pre-API Output

Produce a complete Markdown input summary table with Rationale / Assumption column before invoking the API. Ensure the table is complete and consistent. Allow the user to confirm or adjust inputs before proceeding.

Tool Invocation

Invoke estimateCosysmo with all validated parameters. Verify deterministic parameter order and log the resolved URL in the transcript. Parse returned HTML: extract Effort (PM), Schedule (Months), Cost (USD), Total Size (ENR), Effort Distribution table, and SVG pie chart.

Provenance

Always append the exact calculation source URL so the user can independently verify, modify, and rerun the estimate. Never fabricate results. Handle tool errors gracefully.

COSYSMO API Schema

- OpenAPI specification in YAML defines the formal interface contract between the GPT and COSYSMO tool.
- Specifies server endpoint, named operation, typed parameters, and expected response format.
- Parameter constraints (e.g., enumerated cost driver levels) enforce valid inputs before the API call is made.

```
openapi: 3.1.0
info:
  title: COSYSMO Estimation API
  version: 1.0.0
servers:
  - url: https://softwarecost.org/tools/
paths:
  /cosysmo_ai:
    get:
      operationId: estimateCosysmo
      summary: Estimate cost, schedule, and effort using the COSYSMO model
      parameters:
        - in: query
          name: System_Requirements_Easy
          schema: { type: number }
        - in: query
          name: System_Requirements_Nominal
          schema: { type: number }
        ...
        - in: query
          name: Tool_Support
          schema: { type: string, enum: [Very_Low, Low, Nominal, High, Very_High] }

      responses:
        '200':
          description: HTML with COSYSMO estimation results
          content:
            text/html:
              schema:
                type: string
```

COSYSMO GPT

Access at <https://chatgpt.com/g/g-68a4f33c568881919f1edf346af27e0b-cosysmo-gpt>



COSYSMO GPT

By Ray Madachy &

Trusted COSYSMO effort prediction from softwarecost.org. Avoid model hallucinations, defective calculations, or approximations.

What is the systems engineering effort for 100 nominal...

What is the estimated systems engineering effort...

Example Estimate 1

What is the systems engineering effort for 300 nominal requirements, technology risk = low, and tool support = high?

Talked to softwarecost.org ▾
Here's the COSYSMO estimation for your project parameters.

✔ Input Summary

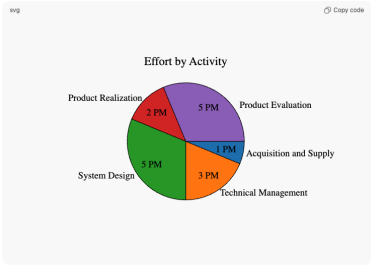
Input Category	Parameter	Value	Rationale / Assumption
System Size	System Requirements (Nominal)	300	Medium-scale system with moderate functional requirements.
Cost Drivers	Technology Risk	Low	Technologies are mature with low uncertainty.
	Tool Support	High	Strong systems engineering toolchain support.
Other	AI Usage	Nominal	Typical automation support, not AI-driven.
Labor Rate	Cost per PM	\$20,000	Standard DoD/Navy systems engineering rate.

Example Estimate 1 (Cont.)

- Estimation Results**
- **Effort:** 15.9 Person-Months
 - **Schedule:** 3.7 Months
 - **Cost:** \$318,234
 - **Total Size:** 300 Equivalent Nominal Requirements

Effort Distribution (Person-Months)

Phase / Activity	Conceptualize	Develop	Operational Test & Evaluation	Transition to Operation
Acquisition and Supply	0.2	0.3	0.1	0.0
Technical Management	0.8	1.4	1.0	0.6
System Design	1.9	2.2	1.0	0.5
Product Realization	0.4	1.0	1.0	0.8
Product Evaluation	1.1	1.6	2.4	1.0



Deterministic COSYSMO API Endpoint

- Access at https://softwarecost.org/tools/cosysmo_ai
- Verify inputs, modify assumptions

Constructive Systems Engineering Cost Model (COSYSMO)

This version of COSYSMO incorporates the new cost factor AI Usage with provisional effort multipliers. Setting AI Usage to Very Low produces equivalent estimates to COSYSMO versions 1 and 2 when other inputs are the same.

System Size

	Easy	Nominal	Difficult
System Requirements	0	250	180
System Interfaces	0	40	0
Critical Algorithms	0	15	0
Operational Scenarios	0	20	0

Cost Factors

Requirements Understanding	High	Documentation	Nominal	AI Usage	Nominal
Architecture Understanding	High	Diversity of Installations/Platforms	High	Personnel Experience/Continuity	Nominal
Level of Service Requirements	High	Recursive Levels in Design	Nominal	Process Capability	High
Migration Complexity	Nominal	Stakeholder Team Cohesion	High	Multisite Coordination	Nominal
Technology Risk	High	Personnel/Team Capability	High	Tool Support	Nominal

Labor Rate

Cost per Person-Month (Dollars) 20000

Calculate

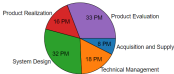
Results

Effort = 107.9 Person-Months
 Schedule = 7.0 Months
 Cost = \$2,157,130
 Total Size = 1611.9 Equivalent Nominal Requirements

Effort Distribution (Person-Months)

Phase / Activity	Conceptualize	Develop	Operational Test and Evaluation	Transition to Operation
Acquisition and Supply	1.1	2.2	0.5	0.3
Technical Management	5.4	8.7	6.5	4.3
System Design	12.9	15.1	6.5	3.2
Product Realization	2.7	6.5	6.5	5.4
Product Evaluation	7.5	10.8	16.2	6.5

Effort by Activity



Activity	Effort (PM)
Product Realization	16 PM
Product Evaluation	33 PM
Acquisition and Supply	8 PM
Technical Management	18 PM
System Design	32 PM

Example Estimate 2

What is the estimated systems engineering effort for a representative NIWC C4I project with 250 nominal requirements and 180 difficult requirements? Apply a realistic Navy C4I profile for the cost driver ratings.

Talked to softwarecost.org ▾

Here is the COSYSMO estimate for your **representative NIWC C4I project**:

Input Summary

Input Category	Parameter	Value	Rationale / Assumption	
System Size	System Requirements (Nominal)	250	Medium-scale C4I system with typical functional and performance requirements.	
	System Requirements (Difficult)	180	Complex, high-criticality and interface-heavy requirements.	
	Interfaces (Nominal)	40	Moderate integration with networks and external systems.	
	Critical Algorithms (Nominal)	15	Includes moderate algorithmic complexity (fusion, encryption, routing).	
	Operational Scenarios (Nominal)	20	Representative set of use cases across operational contexts.	
Cost Drivers	Requirements Understanding	High	Stakeholders have strong domain knowledge and clear mission definition.	
	Architecture Understanding	High	Experienced with Navy reference architectures.	

Example Estimate 2 (Cont.)

	Migration Complexity	Nominal	Evolutionary upgrades to existing systems.
	Technology Risk	High	Emerging AI/ML, 5G, and cyber-resilient components.
	Documentation	Nominal	Baseline SE documentation per Navy standards.
	Diversity of Installations/Platforms	High	Deployed across shore, shipboard, and joint environments.
	Recursive Levels in Design	Nominal	Standard layered architecture, limited recursion.
	Stakeholder Team Cohesion	High	Strong Navy-contractor collaboration.
	Personnel/Team Capability	High	Experienced systems engineers with C4I expertise.
	Personnel Experience Continuity	Nominal	Some turnover expected over the program lifecycle.
	Process Capability	High	Mature systems engineering processes.
	Multisite Coordination	Nominal	Primarily co-located teams.
	Tool Support	Nominal	Standard SE tools used (DOORS, SysML, MBSE limited).
Other	AI Usage	Nominal	Some automation support, not a primary project driver.
Labor Rate	Cost per PM	\$20,000	Typical DoD/Navy SE labor cost.

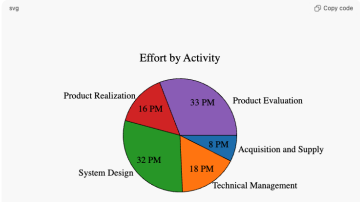
Example Estimate 2 (Cont.)

COSYSMO Results

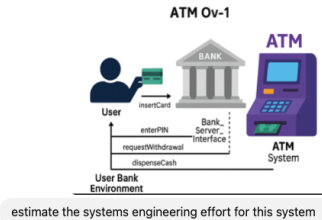
- Effort: 107.9 Person-Months
- Schedule: 7.0 Months
- Cost: \$2,157,130
- Total Size: 1,611.5 Equivalent Nominal Requirements

Effort Distribution (Person-Months)

Phase / Activity	Conceptualize	Develop	Operational Test & Evaluation	Transition to Operation
Acquisition & Supply	1.1	2.2	0.5	0.3
Technical Management	5.4	9.7	6.5	4.3
System Design	12.9	15.1	6.5	3.2
Product Realization	2.7	6.5	6.5	5.4
Product Evaluation	7.5	10.8	16.2	6.5



Example Estimate 3



Excellent — this ATM system operational view (OV-1) is a classic example for a **small, embedded financial transaction system** that interfaces with a bank backend.

Let's create a **COSYSMO-based estimation** for this ATM system.

Step 1: Interpret the System Context

From the OV-1:

- The **ATM System** includes both hardware and software components.
- The **Bank Server Interface** is an external system connection.
- The **User** interacts with the system through simple operations: card insertion, PIN entry, withdrawal request, and cash dispense.

Example Estimate 3 (Cont.)

Step 2: Define System Size Inputs

Category	Parameter	Value	Rationale / Assumption
System Size	System Requirements (Nominal)	80	Basic functional/performance requirements (authentication, transaction, error handling).
	Interfaces (Nominal)	5	ATM ↔ Bank Server, ATM ↔ User I/O, ATM ↔ Cash Dispenser, Card Reader, Network Gateway.
	Critical Algorithms (Nominal)	3	Includes encryption/decryption, PIN validation, transaction verification.
	Operational Scenarios (Nominal)	5	Insert Card, Enter PIN, Withdraw Cash, Balance Inquiry, Transaction Cancel.

Step 3: Cost Driver Inputs (defaulting to *Nominal* unless noted)

Category	Parameter	Value	Rationale / Assumption
Cost Drivers	Requirements Understanding	High	Clear financial domain and well-defined regulatory requirements.
	Architecture Understanding	High	Standard ATM architecture (modular with secure transaction channel).
	Level of Service Requirements	High	Requires high security, reliability, and availability.
	Migration Complexity	Low	New system or upgrade of existing model; low legacy constraints.
	Technology Risk	Nominal	Uses mature hardware and software technologies.

Example Estimate 3 (Cont.)

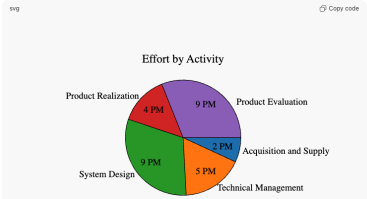
COSYSMO Results

Effort: 29.3 Person-Months
Schedule: 4.6 Months
Cost: \$585,797
Total Size: 178.3 Equivalent Nominal Requirements

Effort Distribution (Person-Months)

Phase / Activity	Conceptualize	Develop	Operational Test & Eval	Transition to Operation
Acquisition & Supply	0.3	0.6	0.1	0.1
Technical Management	1.5	2.6	1.8	1.2
System Design	3.5	4.1	1.8	0.9
Product Realization	0.7	1.8	1.8	1.5
Product Evaluation	2.1	2.9	4.4	1.8

Effort by Activity (Pie Chart)



Live Demonstrations

Play along at this URL: <https://chatgpt.com/g/g-68a4f33c568881919f1edf346af27e0b-cosysmo-gpt>



Verification and Governance

- Maintain COSYSMO model version, calibration basis, API schema, and assistant instructions under configuration control.
- Test equivalence classes for size driver combinations, cost-factor ratings, labor rates, and AI Usage settings.
- Compare API outputs against reference COSYSMO runs.
- Log input configurations and returned results for audit.
- Treat prompt instructions, schemas, and knowledge bases as controlled engineering artifacts.

Results and Discussion

- Effort, schedule, and cost estimates consistently match trusted COSYSMO outputs.
- Deterministic delegation removes hallucination risk from computation while retaining conversational accessibility.
- Smart input inference reduces user burden; narrative descriptions are translated into complete, reviewable parameter sets.
- Improved transparency via estimate tables with input rationales, outputs, and visualizations.
- Structured provenance (source URL, interpreted inputs, assumptions) improves auditability and organizational trust.
- Traceability of inputs through assumptions to outputs simplifies regression testing and audit.

Conclusions

- Deterministic delegation strengthens trust in AI for quantitative model problems.
- LLM is constrained to interpretation, orchestration, and presentation; authoritative computation remains in validated external tools.
- Smart input inference with multi-modal Generative AI lowers barriers to use.
- Together, these support practical, trustworthy AI systems for cost modeling and other quantitative applications in systems and software engineering.

Future Work

- Extend deterministic delegation to additional cost models including COCOMO II.
- Evaluate with practitioners in real-world project settings, specifically NIWC.
- Support project risk management by augmenting Expert COSYSMO [5] to incorporate AI-based risk assessment in conjunction with cost estimation.
- Expand smart input inference with more robust instructions and knowledge base.
- Apply formal, automated testing with input and output data equivalence partitions.
- Extend to broader digital engineering environments and other LLM platforms beyond OpenAI.

References

- [1] R. Madachy. “Trustworthy AI for Quantitative Systems Engineering via Deterministic Delegation to Verified Tools”. In: *INCOSE Insight* (2026).
- [2] R. Valerdi. “The Constructive Systems Engineering Cost Model (COSYSMO)”. PhD thesis. Los Angeles, CA: University of Southern California, 2005.
- [3] R. Madachy. *COSYSMO GPT*. <https://chatgpt.com/g/g-68a4f33c568881919f1edf346af27e0b-cosysmo-gpt>. 2025.
- [4] R. Madachy. *COSYSMO Tool with AI Usage*. http://softwarecost.org/tools/COSYSMO_AI. 2025.
- [5] R. Madachy and R. Valerdi. “Expert COSYSMO Systems Engineering Cost Model and Risk Advisor”. In: *Handbook of Military and Defense Operations Research*. Ed. by N. Scala and